Torneo concurrente de Komikan

Informe de diseño, sincronización y evaluación

Universidad Diego Portales | Sección 3 | Profesor: Cristian Villavicencio
Estudiante: Yeny Díaz | 03 de Noviembre de 2026

Repositorio: https://github.com/JenDaz10/T2_OS

1. Diseño del sistema y recursos compartidos

El programa, desarrollado en **C++17 con POSIX**, recibe `./torneo N M [config.txt]`. Simula la llegada de N clientes, forma parejas y mantiene como máximo M partidas activas. En cada pareja, un cliente representa al Puma y el otro al conjunto de Perros. Cada mesa dispone de un **juez** y de **13 hilos de fichas** (un Puma y doce Perros). Los hilos de una misma mesa comparten su tablero de 25 casillas y 3 posiciones del trono; las distintas mesas tienen tableros separados.

Se dividió el proyecto en módulos: `config` lee los parámetros; `tablero` representa las conexiones entre casillas; `juez` comprueba y aplica las jugadas; `servidor` recibe y empareja clientes; y `main` inicia y cierra el torneo.

2. Esquema de sincronización

- **Semáforo contador (`sem_t`):** comienza con M permisos, uno por mesa. Antes de crear una partida, el servidor realiza `sem_wait`; si las mesas están ocupadas, espera sin consumir CPU de forma continua. Al concluir el juez, se hace `sem_post` para devolver el cupo.
- **Mutex del servidor (`pthread_mutex_t`):** protege la cola de clientes y los cambios de estado compartidos. Las **variables de condición** permiten esperar la llegada de una pareja o el término de una partida sin preguntar constantemente.
- **Mutex por partida:** protege el tablero, la cola de solicitudes y los contadores. Las fichas piensan y proponen movimientos de manera independiente, con retardos aleatorios. **Solo el juez valida nuevamente y aplica** cada solicitud mientras mantiene este mutex.
- **Condiciones del juez:** lo despiertan cuando llegan jugadas y permiten avisar a las fichas cuando su solicitud fue procesada. Un mutex adicional evita que las impresiones de diferentes mesas se mezclen en consola.

Ejemplo de colisión. Si dos Perros solicitan la misma casilla libre, el juez procesa una petición y ocupa esa posición. Al revisar la siguiente, ya no está disponible: la segunda jugada se rechaza. La comprobación y la actualización ocurren dentro de la misma sección crítica, por lo que no pueden quedar dos fichas en una sola casilla.

3. Reglas y cierre de partidas

El juez revisa los movimientos adyacentes, los destinos vacíos y los saltos de captura. El Puma gana al capturar seis Perros; los Perros ganan si el Puma no tiene movimientos válidos, incluidos los saltos. También se aplican límites configurables de tiempo y movimientos para terminar partidas que se prolonguen demasiado.

Al recibir `Ctrl+C`, un hilo espera `SIGINT` mediante `sigwait` y solicita detener nuevas partidas. Los jueces y las fichas reciben el aviso; se espera a que terminen con `pthread_join` y después se liberan mutexes, condiciones y semáforos. El programa también permite **simular** el abandono de un cliente y declarar ganador al oponente.

4. Riesgos de concurrencia y prevención

- **Condición de carrera:** dos fichas podrían ver una casilla libre e intentar ocuparla a la vez. Se evita porque solo el juez modifica el tablero y comprueba cada movimiento bajo el mutex de su mesa.
- **Deadlock (bloqueo mutuo):** Puede ocurrir cuando dos hilos quedan esperando recursos que el otro mantiene bloqueados. Para prevenirlo, se mantiene un orden de adquisición de mutex: el servidor puede bloquear primero su mutex y después el de una partida, mientras que el juez libera el mutex de la partida antes de solicitar el del servidor. De esta manera, se evita una espera circular entre ambos.
- **Livelock (actividad sin progreso):** las fichas podrían insistir en jugadas que dejaron de ser válidas por otros movimientos. Los retardos aleatorios, la elección de nuevas jugadas y la revalidación ayudan a que la partida avance; los límites de tiempo y movimientos impiden una ejecución interminable.
- **Starvation (espera excesiva):** una ficha podría tener menos oportunidades que otras. Las solicitudes del juez se procesan mediante una cola y las esperas son pasivas, aunque no se garantiza un reparto perfectamente justo porque el planificador decide cuándo ejecuta cada hilo.

5. Evaluación de la solución

Se eligió **un mutex por partida** porque resulta más simple de implementar y comprobar que usar un bloqueo distinto para cada casilla. La ventaja principal es la consistencia del tablero. Como contrapartida, los movimientos de una mesa deben validarse **uno a uno**, por lo que no se actualizan casillas en paralelo dentro de esa partida. Las mesas diferentes sí pueden avanzar simultáneamente, hasta el límite M del semáforo.

En cuanto al rendimiento, cada mesa procesa sus movimientos uno a uno, ya que solamente su juez puede actualizar el tablero. Esto limita la cantidad de movimientos procesados dentro de una partida, pero permite mantener la información consistente. Como las mesas funcionan de manera independiente, varias partidas pueden ejecutarse al mismo tiempo, hasta el máximo establecido por M.

Además, la espera pasiva evita gastar CPU consultando continuamente, mientras que los retardos reducen la frecuencia de intentos y el tablero se imprime periódicamente para no saturar la consola. Por estas razones, la solución prioriza la **correctitud** sobre la cantidad de movimientos procesados por segundo. No se midió la latencia exacta ni el *throughput*; las pruebas verifican principalmente el funcionamiento y los límites de concurrencia.

6. Pruebas y alcance

Prueba	**Resultado observado**
20 clientes, máximo 2 mesas	10 partidas; máximo simultáneo observado: 2.
50 clientes, máximo 4 mesas	25 partidas; máximo simultáneo observado: 4.
3 clientes, máximo 1 mesa	1 partida; un cliente quedó sin pareja.
Movimientos y estados del juego	Pruebas de saltos, trono, colisiones, jugadas inválidas y cerco del Puma aprobadas.
Abandono y `SIGINT`	Abandonos simulados detectados y cancelación ordenada comprobada.

Límites de la implementación. Los clientes se representan mediante hilos del mismo proceso, no conexiones de red; los abandonos comprobados son simulados. Un fallo fatal de todo el proceso requeriría un diseño más aislado para recuperarse. La semilla hace reproducibles las decisiones aleatorias, pero no el orden de ejecución de los hilos. El cierre correcto observado tampoco demuestra por sí solo ausencia absoluta de fugas en todas las condiciones.

Conclusión. El semáforo controla cuántas mesas funcionan, los mutexes protegen los datos compartidos y cada juez asegura que los movimientos aplicados respeten el estado actual de su tablero.